Submitted by: Yuriy Rusanov – 340915149, Elad Orpaz - 206216517

Deep Learning

Exercise 3

1. Data Preparation and Preprocessing:

1.1 Data Loading

- **Lyrics Data:** The lyrics were provided in a .csv file, containing columns for artist, song title, and the lyrics themselves. Training data consisted of 600 songs, and the test set contained 5 songs.
- **MIDI Data:** MIDI files corresponding to the songs were provided in a .zip archive. The pretty_midi library was utilized for parsing and analyzing these files.

1.2. Lyrics Preprocessing and Tokenization

- **Lowercase Conversion:** All text was converted to lowercase to ensure consistency.
- **Line Splitting:** Lyrics, often formatted with '&' as a line delimiter in the raw data, were split into individual lines.
- **Tokenization:** Each line was tokenized into words using a regular expression *(r"\b\w+(?:'\w+)*\b")* designed to capture whole words, including those with apostrophes (e.g., "don't"). Empty lines after tokenization were skipped.

The result was a list of lists, where each inner list represents a line of tokenized words. The training set yielded **23659** tokenized lines and test set **189** lines.

1.3. Vocabulary Building

A vocabulary was constructed from the tokenized training lyrics:

- **Word Counts:** A frequency count of all words in the training set was performed.
- **Special Tokens:** Four special tokens were added to the vocabulary:
  - **<PAD>:** For padding sequences.
  - **<NEXT LINE>:** To signify the end of a lyrical line.
  - **<START>:** To mark the beginning of a song's lyrics.
  - **<END>:** To mark the end of a song's lyrics.
  - **<UNK>:** Unknown word not presented in vocabulary
- **Vocabulary Creation:** All unique words were added to the vocabulary, and each word was mapped to a unique integer index. The resulting vocabulary size was **7385**.

1.4. Word Embeddings (Word2Vec)

To represent words as dense vectors, we took pretrained Word2Vec model. Eech word that was not presented in pretrained model was initialized randomly.

- **Model:** *word2vec-google-news-30***0**

- **Embedding Matrix:** An embedding matrix of shape (vocab_size, embedding_dim) was created.
    - For words presented in pretrained Word2Vec model's vocabulary (**6713** words), their respective vectors were used.
    - The special tokens (<NEXT LINE>, <START>, etc.) and words which were not presentend in model's vocabulary (**672** words) were initialized with random vectors drawn from a uniform distribution U(-0.25, 0.25).
    - The <PAD> token's vector remained zeros.

  The matrix final shape (7385,300), it was used to initialize the embedding layer in our neural network models.

2. Approach 1: Simple Model - Global Melody Features

The first approach involved a model that considers global features of the melody, providing a general musical context for lyrics generation.

2.1. Dataset for Simple Model

- For this model, a fixed set of global features was extracted from each MIDI file:
    - **Tempo:** Mean tempo of the song (derived from tempo changes).
    - **Average Pitch:** Mean pitch of all notes in the song.
    - **Pitch Standard Deviation:** Standard deviation of pitches.
    - **Pitch Range:** Difference between maximum and minimum pitch.
    - **Average Duration:** Mean duration of notes.
    - **Average Velocity:** Mean velocity (loudness) of notes.
    - **Note Density:** Total number of notes divided by the song's total duration.

- **Sequence Preparation:** For each song, lyrics were tokenized, and special tokens (<START>, <NEXT LINE>, <END>) were added. <START> was prepended, <NEXT LINE> was appended after each actual line, and the final <NEXT LINE> was replaced by <END>.
- **Input-Target Pairs:** The entire song's token sequence was used to create input-target pairs. For each token word_i in the sequence, the input was word_i and the target was word_{i+1}.
- The full dataset was split into training (80%) and validation (20%) sets.
  We experimented with smaller validation splits but observed that evaluation on such subsets was unstable and did not accurately reflect the model's true performance.
  DataLoaders were configured with a batch size of 64. The training DataLoader was set to shuffle the data to ensure better generalization.
    - Train dataset size: 143243 samples.
    - Validation dataset size: 35811 samples.

2.2. Simple Model Architecture

We chose LSTM instead of GRU because it remembers information over longer sequences better, which helps when working with both lyrics and melody. It also gives more control over what to keep or forget, making it better for handling complex patterns.

The model is an LSTM-based network:

- **Inputs:** Current word_input (index) and global melody_features (vector).
- **Embedding Layer:** nn.Embedding(vocab_size, embedding_dim) initialized with the pretrained Word2Vec embedding matrix. It converts the word_input index into a dense vector.
- **Melody Projection:** nn.Linear(melody_dim, embedding_dim) projects the 7-dimensional melody feature vector to the same dimension as the word embeddings (300). This allows for easy concatenation.
- **Concatenation:** The word embedding and the projected melody embedding are concatenated along their feature dimension, resulting in a vector of size embedding_dim * 2. This combined vector is unsqueezed to add a sequence length dimension of 1, making it suitable for LSTM input (batch, 1, embedding_dim * 2).
- **LSTM Layer:** nn.LSTM(input_size=embedding_dim * 2, hidden_size=512, num_layers=4, batch_first=True, dropout=0.3). This layer processes the sequence of (word + melody) features.
- **Dropout Layer:** nn.Dropout(0.3) applied to the LSTM output for regularization.
- **Fully Connected (FC) Layer:** nn.Linear(hidden_dim, vocab_size) maps the LSTM's hidden state output to a score distribution over the entire vocabulary, predicting the next word.
- **Outputs:** Raw logits for the next word and the updated LSTM hidden state.

2.3. Simple Model Training

- **Loss Function:** nn.CrossEntropyLoss, suitable for multi-class classification (predicting the next word).

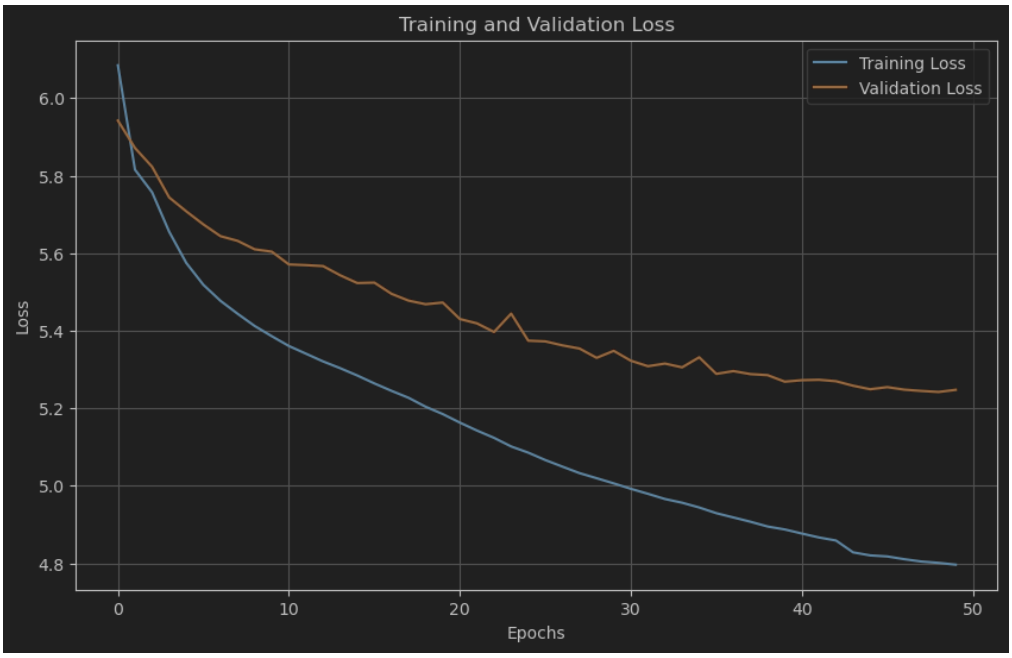
- **Optimizer:** optim.Adam with a learning rate of $5 * 10^{-5}$.

- **Learning Rate Scheduler:** optim.lr_scheduler.ReduceLROnPlateau was used to reduce the learning rate by a factor of 0.5 if the validation loss did not improve for 2 consecutive epochs.

- **Epochs:** The model was trained for 50 epochs.

- **Gradient Clipping:** Gradients were clipped to a maximum norm of 5.0 (torch.nn.utils.clip_grad_norm_) to prevent exploding gradients.

- **Best Model Saving:** The state of the model with the lowest validation loss was saved during training.



Best model training loss: **4.8012**, validation loss: **5.2421**

The learning curves indicate that while training loss continued to decrease, the validation loss plateaued around the point. It suggests the model had reached its learning capacity for the given architecture and data, and further training leads to overfitting

2.4 Simple Model Lyrics Generation

Starting Words: Three different starting words were used for each test melody:

- <START> - context not provided, the model should rely only on melody features at the beginning.
- Love – general positive context provided.
- Hate – general negative context provided.
- Song's original word – to give more context to model.

Generation Loop:

1. The model takes the current word and the (constant) melody features to predict the next word's probability distribution.
2. **Sampling:** The next word is chosen by sampling from this distribution using torch.multinomial. A temperature parameter (set to 0.9 for testing) was used to scale the logits before applying the softmax and sampling.
3. The chosen word becomes the input for the next step.
4. The process continues until an <END> token is generated, or a max_length (set to average 250 words per song) is reached.

Formatting: The generated sequence of tokens (words) was formatted:

- <START> and <END> tokens were removed.
- <NEXT LINE> tokens were converted to (\n). Consecutive newlines were collapsed.
- The final output is a string of generated lyrics.

Examples of generated lyrics:

Song: the_bangles - eternal_flame. Starting word: '<START>'

you you i what let i'm

and

you about

and

i

tomorrow

yeah is

when

what just it

i

is

is

yeah on

to

that faith

i've

you...

Analysis:

The output has several characteristics:

- **Fragmented Lines:** The model tends to generate very short lines, often consisting of only one or a few words (e.g., "and", "i", "to"). **Low Coherence:** The sequence of words lacks clear semantic connection, making the lyrics difficult to understand. For instance, phrases like "you you i what let i'm" or "yeah is when what just it" do not form coherent sentences.

- **Repetition:** There is evidence of word repetition (e.g., "you you", "is is"), which can be indicative of the model struggling to generate diverse and meaningful content. It happens possible because these words are presented more often in the dataset.

- **Limited Contextual Understanding:** The use of global melody features appears insufficient to provide fine-grained, word-by-word contextual guidance, leading to a somewhat generic and disconnected output. The model seems to struggle to build upon previous words to form longer, meaningful phrases.

3. Approach 2: Advanced Model - Time-Aware Melody Features

The second approach aimed to integrate melody information more dynamically by considering musical features that are temporally aligned with the words being generated.

3.1 Dataset for Advanced Model

- This model uses melody features extracted for specific time windows corresponding to words or sequences of words.
    - **Line Duration Allocation (allocate_line_durations):** The total duration of a MIDI file was distributed among its lyric lines based on the number of characters in each line. This provided an estimated duration for each line.
    - **Word Timing Estimation (estimate_word_durations):** Within each line, the estimated line duration was distributed among its words based on the number of letters in each word. This yielded estimated start and end times for each word.
    - **Feature Extraction (extract_time_based_features):** For each word's estimated time window (start_time, end_time), extended by a context of +/- 5 seconds, the following features were extracted from the MIDI data:
        - **pitch_mean:** Mean pitch of notes within the window.
        - **pitch_std:** Standard deviation of pitches within the window.
        - **velocity_mean:** Mean velocity of notes within the window.
        - **duration_mean:** Mean duration of notes within the window.
        - **note_density:** Number of notes within the window. If no notes were found in a window, these features were set to 0.
    - **Feature Normalization:** The extracted features were normalized (e.g., pitch mean / 127.0, pitch std / 12.0, velocity mean / 127.0). The final feature vector for each word timing was 5-dimensional
- **Context Size:** A context_size of 16 words was used.
- **Sequence Preparation:**
    - Lyrics were tokenized similarly to the simple model.
    - Each song's lyrics sequence was prepended with context_size <START> tokens. Corresponding dummy musical features (all zeros) were used for these <START> tokens.
    - Time-based musical features were extracted for each actual word in the song as described above. <NEXT LINE> tokens reused the features of the preceding word. The final <NEXT LINE> became <END>.

- **Input-Target Pairs:** A sliding window of context_size was moved across the full token sequence. For each window:
    - **context_words**: A sequence of context_size word indices.
    - **context_features:** A sequence of context_size corresponding 5-dimensional musical feature vectors.
    - **target_word:** The word index immediately following the context window.
    - **target_features:** The 5-dimensional musical feature vector corresponding to the *timing* of the target_word.

The dataset was split into training (80%) and validation (20%). DataLoaders used a batch size of 64.

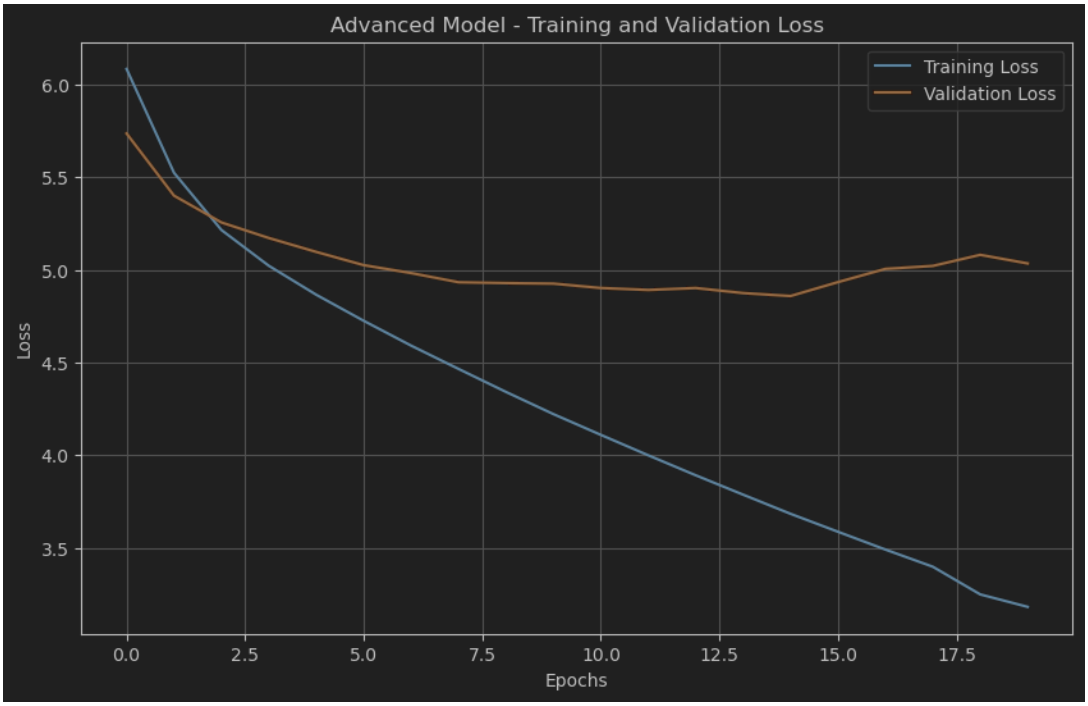
3.2 Advanced Model Architecture

This model uses an LSTM with attention and incorporates time-specific musical features for both context and target words.

- **Inputs:** context_words (indices), context_features (vectors for context), target_features (vector for the timing of the word to be predicted).
- **Embedding Layer:** nn.Embedding(vocab_size, embedding_dim) initialized with pretrained Word2Vec embeddings.
- **Feature Encoder:** A nn.Sequential module processes musical features. It consists of two linear layers with ReLU activations and Dropout, mapping the 5-dimensional feature vector to hidden_dim // 2. This encoder is used for both context features and target features.
- **Context Processing:**
    1. Word embeddings for context_words are obtained.
    2. Musical features for context_features are encoded using the feature encoder.
    3. The word embeddings and encoded music features are concatenated for each item in the context sequence. The combined dimension is embedding_dim + (hidden_dim // 2).
    4. LSTM Layer: nn.LSTM(input_size=embedding_dim + (hidden_dim // 2), hidden_size=512, num_layers=4, batch_first=True, dropout=0.3) processes this combined sequence.
    5. Attention Layer: nn.MultiheadAttention(embed_dim=hidden_dim, num_heads=4, batch_first=True) is applied to the LSTM output. This allows the model to weigh the importance of different context words.
- **Combining Information:**
    1. The final hidden state of the LSTM (h_n[-1]).
    2. The attention output corresponding to the last element of the context sequence (attn_output[:, -1, :]).
    3. These two are concatenated.
- **Target Feature Integration:** The target_features (musical features for the timing of the word to be predicted) are encoded using the same feature encoder.
- **Final Representation:** The (LSTM hidden + attention output) representation is concatenated with the encoded target_features. The total dimension of this combined vector is hidden_dim + hidden_dim + (hidden_dim // 2).
- **Output FC Layers:** A nn.Sequential module with a linear layer, ReLU, Dropout, and a final linear layer maps this comprehensive representation to scores over the vocab_size.
- **Output:** Raw logits for the next word.

3.3 Advanced Model Training

- **Loss Function:** nn.CrossEntropyLoss.
- **Optimizer:** optim.Adam with a learning rate of $1 * 10^{-4}$.
- **Learning Rate Scheduler:** optim.lr_scheduler.ReduceLROnPlateau (patience=2, factor=0.5).
- **Epochs:** The model was trained for 20 epochs.
- **Gradient Clipping:** Max norm of 5.0.
- **Best Model Saving:** Based on validation loss.



Best model traninig loss: 3.6836, validation loss: 4.8750.

This reduction in validation loss suggests that the Advanced Model, with its time-aware features and attention mechanism, was able to learn more effective representations and capture more complex patterns within the data

3.4 Advanced Model Lyrics Generation

Context Preparation: The input start_words were padded with <START> tokens to match the model's context_size.

Timing Estimation for Generation:

- The total duration of the MIDI and the max_length of lyrics to generate were used to estimate an average word_duration.
- An initial current_time cursor was set.
- For the initial context words, musical features were extracted based on their estimated timings.
- **Generation Loop:**
    1. For the next word to be predicted, its timing (start and end) was estimated by advancing current_time by word_duration (with some added randomness: random.uniform(-2, 2) seconds to the duration).
    2. Musical features for this *target timing* were extracted.
    3. The model predicted the next word based on the current word context, context musical features, and target timing musical features.
    4. Sampling with temperature (0.9 for testing) was used.
    5. The chosen word was added to the output, and the context (words and their features) was updated by sliding the window. current_time was advanced.
    6. This continued until <END> or max_length (750 words).

Formatting: Similar to the simple model, lyrics were formatted to handle special tokens and newlines.

Examples of generated lyrics:

Song: the_bangles - eternal_flame. Starting word: '<START>'

hi out me all

don't save me come down

put it on i'll try again try

get it out i'll get it

we got you got it

if it is to be to

what are you baby

what baby i have to come enough

ooh khan now what can i come

oh i don't want to show you there right more

cause you need me off in the tank for you instead

she told me if we would sail

the papers got his different friend

for the copa lane light in the sea

his boardwalk

another is die...

Analysis:

The model demonstrates noticeable improvements:

- **Improved Line Structure:** Lines are generally longer and more structured compared to the Simple Model's output (e.g., "don't save me come down", "put it on i'll try again try").
- **Enhanced Semantic Coherence:** While not perfect, the generated text exhibits better local coherence. Phrases appear more "sentence-like," and there's a suggestion of developing ideas (e.g., "cause you need me off in the tank for you instead").
- **Increased Complexity:** The vocabulary and phrasing seem more varied. The model attempts more complex constructions.
- **Emergence of Lyrical Qualities**. While strong, consistent rhymes may not be, the overall textural improvements point towards a better grasp of lyrical forms. For instance, phrases like "get it out i'll get it" or "we got you got it" show some repetitive lyrical patterning. This output suggests that the time-aware melody features and attention mechanism allow the Advanced Model to generate more contextually relevant and structurally sound lyrics.

4. Results and Comparative Analysis

We used the BLEU score to check how similar the lyrics generated by our models were to the original song lyrics. A higher BLEU score means the generated lyrics are more like the originals. The table below shows the BLEU scores for both the Simple and Advanced models, using different starting words for each song:

	Simple Model				Advanced Model				
Starting Word:	<START>	love	hate	song's original first word	<START>	love	hate	song's original first wor	Average Per Song
the bangles - eternal flame	0.0018	0.0074	0.0125	0.0077	0.0202	0.0091	0.0118	0.0265	0.012125
billy joel - honesty	0.0042	0	0	0.0064	0	0.01	0	0.0238	0.00555
cardigans - lovefool	0.0002	0.0002	0	0.0016	0.017	0.0104	0	0.0031	0.0040625
aqua - barbie girl	0	0	0.0001	0.0001	0.0002	0	0	0.0079	0.0010375
blink 182 - all the small things	0.0005	0	0.0039	0.0019	0	0.0082	0.0076	0	0.0027625
Average Per Song	0.00134	0.00152	0.0033	0.00354	0.00748	0.00754	0.00388	0.01226	
Average Per Model			**0.002425**				**0.00779**		

Key Findings:

- **Advanced Model Performed Better:** The Advanced Model consistently got much higher BLEU scores than the Simple Model in all our tests. This matches what we saw when we read the lyrics: the Advanced Model writes lyrics that make more sense and fit the song's context better. This improvement is likely because it uses time-aware melody features and attention, which are more sophisticated ways to understand the music.
- **Starting Word Context Was Important:** Both models achieved their highest BLEU scores when we gave them the song's actual first word as a starting point. This is logical because the original first word provides the strongest and most relevant clue about the song's content, helping the model generate lyrics closer to the real ones. When we used general starting words like <START>, or words with a general feeling like 'love' or 'hate', the models had less specific guidance, which led to lower BLEU scores.
- **Song Style and Vocabulary Affected Scores:** We noticed that BLEU scores varied from song to song. For example, 'The Cardigans - Lovefool,' which uses common words that were likely in our training data, generally received higher BLEU scores. In contrast, a song like 'Aqua - Barbie Girl,' which has unique or informal words (like 'hiya') that might not have been common in the training data, got lower BLEU scores. This shows that the models perform better when the song's vocabulary is similar to what they learned during training.
- **Length of Generated Lyrics:** There was a clear difference in how long the lyrics were. The Simple Model tended to write short lyrics, averaging about 52 words per song. The Advanced Model, however, wrote much longer lyrics, averaging around 120 words. Since the Advanced Model's longer lyrics also generally made more sense, this suggests it's better at continuing a lyrical idea and developing themes within a song.

In summary, looking at metrics and the actual quality of the lyrics, the Advanced Model is a significant step up from the Simple Model. The way it's built allows it to use information from the melody in a more detailed way. This helps it write lyrics that are usually longer, clearer, and sound more like real song lyrics, especially when it's given a good, specific starting word.

You can find all the lyrics generated by the models for each test song and each starting word in the generated_lyrics_results.txt file.